ESP Lab2: Multirate FIR Branching Filter

1. Learning Outcomes

In this lab, you implement a multirate FIR filter system where the filter is implemented efficiently as a kernel filter in the low-frequency domain. The branching filter approach allows an effective way to create a low/high pass pair of filters. The system overall consists of decimator, kernel filter and interpolator in the upper branch and (in the second part of the lab) of delay cascade in the lower branch.

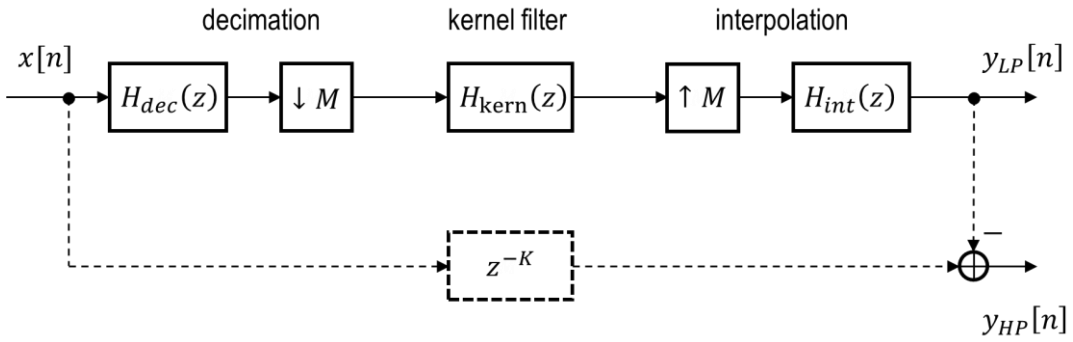


Figure 1: Multirate branching filter to be implemented.

An example magnitude response of a low/high pass branching filter pair is given in Figure 2.

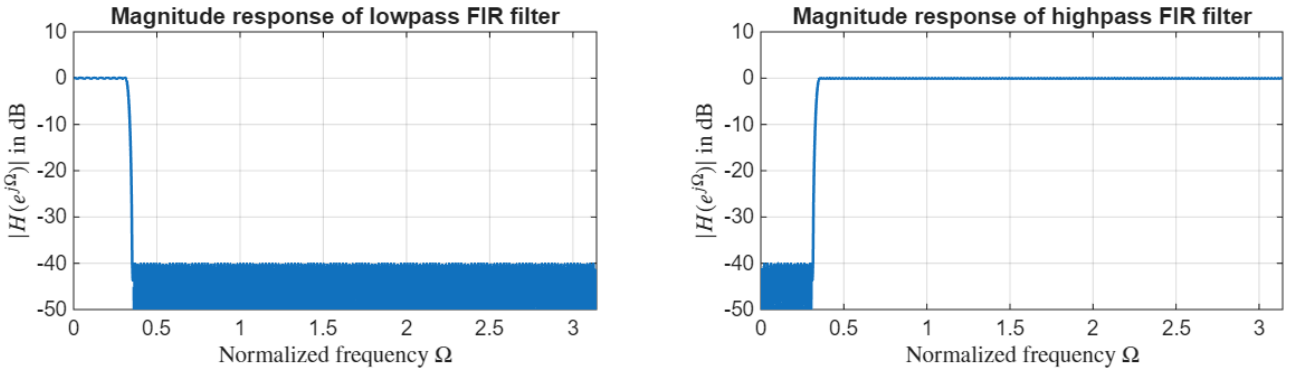


Figure 2: Magnitude responses of a low/high pass branching filter pair

The learning objectives of this lab are to

- understand down-sampling and up-sampling in time and frequency domain and how we cope with aliasing / imaging spectral components,
- design linear-phase FIR high-pass/low-pass branching filters to a given specification,
- derive and parametrize a multirate system with decimator, kernel filter, interpolator and how to choose an optimum factor M (where $L = M$),
- translate a Matlab filter design into a real-time fixed-point C implementation on the DSP, including polyphase representation in the low-frequency domain and distribution of the multirate system to ADC and DAC interrupt handlers,
- verify the correct function of the system using amplitude/frequency responses using oscilloscope and audio analyzer.

2. Task

We are approaching the task of developing a branching filter in three steps: first we implement the desired filter directly, i.e. as a single rate lowpass filter in the high frequency domain and make an estimate of the necessary computational power. Afterwards we optimize the lowpass filter as a multirate filter for which we need to find the optimum M_{opt} and the filter coefficients of the polyphase filter. Eventually we develop the complementary high-pass filter by implementation of the branching filter.

The filter specifications for groups ESPL/01 and ESPL/02 are as follows:

Parameters		**Values**	
		ESPL/01	**ESPL/02**
sampling rates of ADC and DAC		$f_s = 50$ kHz	
lowpass filter	**passband edge**	$f_{\text{stop},LP} = 3100$ Hz	$f_{\text{stop},LP} = 2700$ Hz
	stopband edge	$f_{pass,LP} = 3350$ Hz	$f_{pass,LP} = 2900$ Hz
complementary highpass filter	**stopband edge**	$f_{\text{stop,HP}} = 3100$ Hz	$f_{\text{stop,HP}} = 2700$ Hz
	passband edge	$f_{pass,HP} = 3350$ Hz	$f_{pass,HP} = 2900$ Hz
total passband ripple		$\delta_{pass} = 0.01$ (linear, not dB)	
minimum stopband attenuation		$\delta_{stop} = 40$ dB	
quantization of signals and filter coefficients		16 bit (1Q15 format)	

Table 1: Specification of the filters for groups ESPL/01 and ESPl/02

3. Preparation

3.1 Conventional lowpass filter

- Design a linear-phase FIR low-pass and its complementary high-pass using firpmord() and firpm(). Use the specification above und the script `dec_kernel_int.m` provided in Moodle.
- Plot both magnitude responses in dB for both filters in two different subplots. Verify δ_{pass} and δ_{stop}.
- From the Matlab output, record the filter degree (order) for the single-rate solution.
- Compute the DSP cycles available per sampling period at f_s= 50 kHz and f_{DSP}=456 MHz. Discuss the feasibility of real-time implementation on the DSP without further code optimization.

Add your results, including Matlab code and figures and discussion, to the preparation sheet.

3.2 Multi-rate FIR filter design

The design of the multi-rate system in Figure 1 is carried out in several steps:

- We determine an optimal M_{opt}.
- We decompose the original filter into decimation, interpolation, and kernel filters.
- We determine the delay K of the all-pass filter to obtain a correctly functioning branching filter.

3.2.1 Optimum choice of downsampling/upsampling factor M

Using the specification given above, calculate the optimal value for M_{opt} in Matlab.

Add the exact value for M_{opt} and the nearest integer number to the prep sheet.

Regardless of which M_{opt} you have determined, select M for the lab tasks as given in the following table.

Parameter choice for upsampling and downsampling factor	Values	
	ESPL/01	ESPL/02
M	4	5

Table 2: Parameter choice for upsampling and downsampling factor for groups ESPL/01 and ESPl/02

3.2.2 Decomposition into decimation, interpolation and kernel filter

The decimation filter and the interpolation filter can be chosen identical. Therefore, only the decimation filter and the kernel filter need to be designed. As the passband ripples of each filter add under unfavorable conditions, the ripple of each filter is chosen to be 1/3 of the total required ripple of the original filter.

Determine the theoretical values of

- the passband and stopband edge frequency of the decimation filter,
- the passband and stopband edge frequency of the kernel filter,
- the minimum passband attenuation of kernel and decimation filter.

Using the found values, extend the script `dec_kernel_int.m` with the design of the FIR filters in MATLAB via `firpmord()` and `firpm(...)` and plot the magnitude responses of

- the decimation filter,
- the kernel filter and
- the cascade of all the three filters.

Mind that the polyphase filters of both decimation and interpolation filter are implemented by M polyphase filters (which operate in the low frequency domain) as shown in Figure 3.

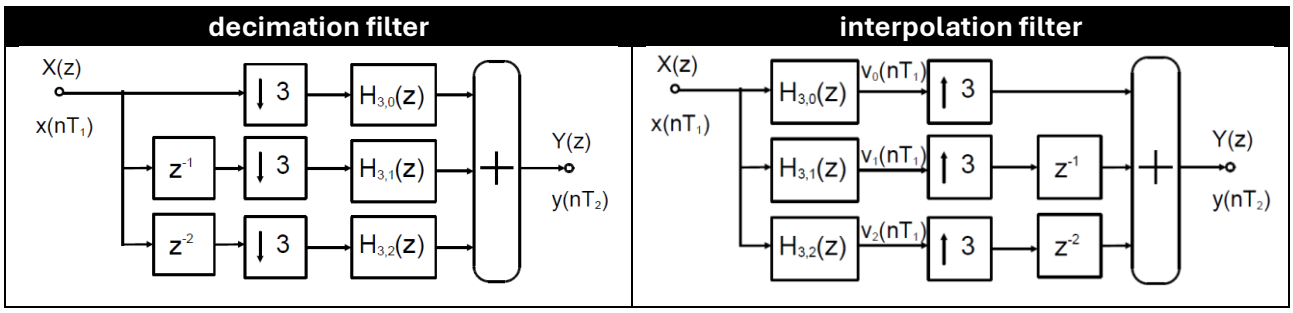


Figure 3: Efficient polyphase filter structures for $M = 3$.

This means that you have to create and export M different sets of filter coefficients, one for each polyphase filter. The slideset in the lecture provides an example Matlab code to implement the structures in Figure 3. Of particular importance is the order in which the input values (arriving at high sampling frequency) are distributed to the polyphase filters $H_{M,\lambda}(z')$ of the decimation filter, or the order in which the values are read out (again at high sampling frequency) from the individual polyphases of the interpolation filter. Relevant information (so called "switch model") was provided in the lecture.

Explain with Matlab simulations (magnitude responses), why you think the results are correct. Mind that decimation filter and interpolation filter are assumed to run on f_s (as M polyphase filters are executed at f_s/M), while the kernel filter runs on f_s/M.

Write the coefficient sets of the filters to the header file `dec_kernel_int.h` using `write_coeff.m`.

Add the filter parameters found above, your Matlab code and the magnitude plots including explanations to the prep sheet. Explain your approach to find the filter parameters and the verification of the results using sketches of the spectra. Do not forget to bring the filter in the header file parameters to the lab

3.3 Comparison of computational effort

Compare the computational effort of the original filter $E_{total,single}$ in Section 3.1 and the multirate filter $E_{total,multi}$ in Section 3.2.2 for the values of $M = \mathrm{L}$ as given in Table *2*. Find the

- the order of decimation / interpolation filter,
- the order of the kernel filter and
- the computational effort $E_{total,single}$ and $E_{total,multi}$.

How probable is a successful real-time implementation of the multirate filter on the DSP? Determine the group delay τ (in samples, assuming the fast sampling frequency f_s) of the conventional filter and compare it to the multirate filter.

Add the calculation of $E_{total,single}$ and $E_{total,multi}$ and the group delay to the prep sheet and explain, whether a real-time implementation is possible.

The preparation is complete. Congratulations! **Please submit the preparation sheet in Moodle before the lab session.**

4. Lab task

In the lab, you will first try to implement the conventional FIR low pass filter as you did in the first ESPL lab session. Afterwards you implement the multirate system of the low-pass filter

with decimation, kernel and interpolation filter on the DSP in C and use the audio analyzer to check that it is functioning correctly. In addition, you will add the branching filter to output a pair of complementary highpass and lowpass filtered signals.

4.1 Implementation of the conventional low-pass filter

Implement the low-pass filter as designed in Section 3.1 in the `intADC()` interrupt handler using the assembler routine `FIR_filter_asm_sc.asm` (details can be found in the lab report for the first ESPL session).

Add the relevant C code of the intADC() interrupt handler and make a note of your observations when implementing the conventional filter on the DSP.

4.2 Implementation of the multirate low-pass filter

Extend the C code for a multirate filter so that decimation filter, interpolation filter (both in polyphase representation) and kernel filters are calculated in the `intADC()` interrupt handler. This is possible because $M = L$ applies and it simplifies the software design.

Use the template `decimate_by_5_polyphase.txt` provided in Moodle (and shown in the appendix) as a starting point for your program. Measure the magnitude response of the multirate filter with the audio analyzer.

Add the final C code and the screenshot of the magnitude response of the multirate filter to the report. Interpret the magnitude response of the multirate filter.

4.3 Implementation of the branching filter

Now add the lower branch, which consists of an allpass with a delay of K samples, to the multirate lowpass filter (see Figure 1). The value of K is to be determined first – what does it depend on?

Implement the delay of the input signal by K cycles in the intADC() interrupt handler and subtract the output of the low-pass filter from the all-pass filter to obtain the high-pass filtered output signal. Output both signals $y_{LP}[n]$ and $y_{HP}[n]$ and measure the magnitude response of the low-pass and high-pass filters.

Add the C code of the allpass branch and the screenshots of the magnitude response of highpass and lowpass filter to the report. Verify that the result is as expected.

Congratulations! **You made it. Please submit with 15 min. after end of the session.**

Appendix

Template for software design of the multirate filter

```
_interrupt void adcInt (void)
{
   static int y1_part1=0,y1_part2=0,y1_part3=0,y1_part4=0,y1_part5=0;

    /*****************************************************************
      read two ADC channels and store in floating point format
    *****************************************************************/
   for (idx=0; idx<1; idx++)
```

```
        {
                sData[idx] = PRU_addaRegs->adc[idx];
        }

        inL = sData[0];                     // use channel 0 only

// counter for ADC interrupt
        if (countAdcInt >= MM)
                countAdcInt = 0;

// Polyphases of decimator filled in the order lambda = 0, 4, 3, 2, 1 and repeating,
// at 0 sum is calculated and input to kernel filter.
// Kernel filter and interpolation filter not yet implemented here.


// distribute computations across 5 output ADC interrupts
//---------------------------------  sample 0 ---------------------------------
   if (countAdcInt == 0) {
                countAdcInt++;
                y1_part1 = FIR_filter_sc( H_filt_50_delays, // compute polyphase FIR out and save
                                b_filt_b0_b5_usw, N_delays_H_filt_50_delays, inL, 15);
        }

        // add all partial results up, y1 must be input to kernel filter at rate f_s/5
        y1 = y1_part1 + y1_part2 + y1_part3 + y1_part4 + y1_part5;

//---------------------------------  sample 1 ---------------------------------
        else if (countAdcInt == 1) {
// for INT handler
                countAdcInt++;
                y1_part2 = FIR_filter_sc ( H_filt_54_delays, // compute polyphase FIR out and save
                                b_filt_b4_b9_usw, N_delays_H_filt_54_delays, delay4, 15);
                delay4 = inL;                                           // now update delay
        }

//---------------------------------  sample 2 ---------------------------------
        else if (countAdcInt == 2) {
// for INT handler
                countAdcInt++;
                y1_part3 = FIR_filter_sc ( H_filt_53_delays, // compute polyphase FIR out and save
                                b_filt_b3_b8_usw, N_delays_H_filt_53_delays, delay3, 15);
                delay3 = inL;                                           // now update delay
        }

//---------------------------------  sample 3 ---------------------------------
        else if (countAdcInt == 3) {
// for INT handler
                countAdcInt++;
                y1_part4 = FIR_filter_sc ( H_filt_52_delays, // compute polyphase FIR out and save
                                b_filt_b2_b7_usw, N_delays_H_filt_52_delays,delay2, 15);
                delay2 = inL;                                           // now update delay
        }

//---------------------------------  sample 4 ---------------------------------
        else if (countAdcInt == 4) {
// for INT handler
                countAdcInt++;
                y1_part5 = FIR_filter_sc ( H_filt_51_delays, // compute polyphase FIR out and save
                                b_filt_b1_b6_usw, N_delays_H_filt_51_delays,delay1, 15);
                delay1 = inL;                                           // now update delay

        }

        return;
}
```